

FoodTally AI Engine

FoodTally is a high-tech nutrition monitoring ecosystem that combines an asynchronous Telegram bot, the power of FastAPI, and Artificial Intelligence to analyze meals via photos.

Architecture & Data Flow

The project implements an **Event-Driven Architecture** using two-way webhooks:

1. **User Side:** The user sends a photo or data via Telegram (aiogram 3.x).
2. **Processing:** The bot forwards the file_id and user metadata to **Make.com**.
3. **AI Analysis:** The Make.com scenario utilizes AI (Gemini/OpenAI) to decompose the dish into ingredients and calculate macronutrients.
4. **Ingestion:** Results are returned to the system via **FastAPI Webhooks** (/make_record, /make_reply, etc.).
5. **Finalization:** The system formats the response using HTML markup, saves the report to **PostgreSQL**, and notifies the user.

Tech Stack

- **Backend:** Python 3.11, FastAPI (Uvicorn).
- **Telegram Framework:** aiogram 3.x (Webhook mode).
- **Database:** PostgreSQL + asyncpg (asynchronous connection pooling).
- **Automation:** Make.com (scenario integration and AI models).
- **Deployment:** Railway.

API Endpoints (FastAPI)

The system acts as an API server for external processors (Make.com):

- POST /webhook — Receives updates from the Telegram Bot API (via dp.feed_webhook_update).
- POST /make_record — Receives data, sends a report to the user, and records it in the database.
- POST /make_reply — Fast response with nutrients and balance assessment (no DB record).
- POST /make_build — Result of the meal builder (includes step-by-step cooking process).
- POST /make_shop_help — Generates a grocery list based on history.

Database Structure & Formatting

Data presentation logic is integrated directly into the database access methods to ensure a consistent UI:

Personal Info (users_info)

Stores user profiles: age, height, weight, gender, goal.

- The `user_personal_info` method returns a ready-to-use visual profile card using `—————` separators.



Meal Reports (`meal_report`)

Meal history: `calories_estimated`, `protein_g`, `fat_g`, `carbs_g`, `balance_assessment`, `products_list`.

- `show_menu_reports`: Returns a formatted block of the latest meal.
- `show_product_list`: Generates a summary list of ingredients for the last 7 days.



Deployment on Railway

1. **Database:** Create a PostgreSQL instance in the Railway dashboard.
2. **Variables:** Configure the following environment variables:
 - `BOT_TOKEN` — Bot token from @BotFather.
 - `DATABASE_URL` — PostgreSQL connection string.
 - `WEBHOOK_URL_RAIL` — Public URL of your service (e.g., `https://your-app.up.railway.app`).
 - `MAKE_WEBHOOK_URL` — Incoming hook URL in Make.com.
3. **Webhook:** Upon startup, the application automatically resets old webhooks and registers the `/webhook` address on the current domain.

Developed for health-conscious users who value speed and precision.